Ders: BLM442 Yapay Sinir Ağları

Proje Adı: Food-11 Veri Seti Üzerinde CNN Tabanlı Derin Öğrenme ile Yemek Görüntü Sınıflandırması

Tarih: 2025–2026 Bahar Dönemi

Proje Ekibi ve Görev Dağılımı

Katkı Alanı	**Sorumlu**
Veri setinin hazırlanması, klasör yapısının oluşturulması, veri artırma (augmentation) yöntemlerinin araştırılması ve uygulanması	Ahmet Emre Oruç
ResNet-50 modelinin MATLAB'a aktarılması, 2 aşamalı eğitim sürecinin tasarlanması ve eğitimin yürütülmesi	Berke Bilgiz
Test sonuçlarının değerlendirilmesi, grafiklerin oluşturulması, rapor yazımı	Oğuzhan Karahan

---

1. Giriş

Bu çalışmada, Food-11 veri seti kullanılarak yemek görüntülerinin otomatik olarak 11 kategoriye sınıflandırılması amacıyla derin öğrenme tabanlı bir sistem geliştirilmiştir. ResNet-50 mimarisi üzerine iki aşamalı transfer learning (aktarım öğrenimi) uygulanmış; veri dengesizliği sınıf ağırlıklandırması, aşırı öğrenme ise veri artırma ve erken durdurma mekanizmaları ile giderilmiştir. Geliştirilen sistem, test setinde **%90,56 doğruluk**, **%91,1 Makro F1** ve **%90,6 Ağırlıklı F1** skoru elde etmiştir.

Görüntü sınıflandırma, derin öğrenmenin en başarılı uygulama alanlarından biridir. Yemek tanıma sistemleri; beslenme takibi, restoran menüsü analizi ve gıda güvenliği gibi pratik uygulamalarda giderek artan bir öneme sahip olmaktadır. Geleneksel makine öğrenimi yöntemleri elle tasarlanan özellik çıkartıcılara dayanırken, Evrişimli Sinir Ağları (CNN) ham piksel verilerinden hiyerarşik özellikler öğrenerek bu sınıflandırma problemlerini çok daha yüksek doğrulukla çözebilmektedir.

Bu çalışmanın amacı, halka açık Food-11 veri setini kullanarak 11 farklı yemek kategorisini birbirinden ayırt edebilen bir CNN modeli geliştirmektir. Modelin sıfırdan eğitilmesi yerine, büyük ölçekli ImageNet veri seti üzerinde önceden eğitilmiş ResNet-50 mimarisinden yararlanılmış ve iki aşamalı ince ayar (fine-tuning) stratejisi uygulanmıştır.

---

2. Kullanılan Araçlar ve Teknolojiler

Araç / Teknoloji	Kullanım Amacı
MATLAB R2023b	Ana geliştirme ortamı
Deep Learning Toolbox	CNN eğitimi, transfer learning
ResNet-50	Önceden eğitilmiş temel model (backbone)
Food-11 Veri Seti	11 sınıflı yemek görüntü veri seti
GPU (varsa)	Hızlı model eğitimi

---

3. Veri Seti

3.1 Genel Bilgi

Food-11, 11 farklı yemek kategorisini içeren, görüntü sınıflandırması için hazırlanmış bir veri setidir. Veri seti üç alt klasöre ayrılmıştır:

training/ → Modelin eğitildiği görüntüler

validation/ → Eğitim sırasında modelin izlendiği görüntüler

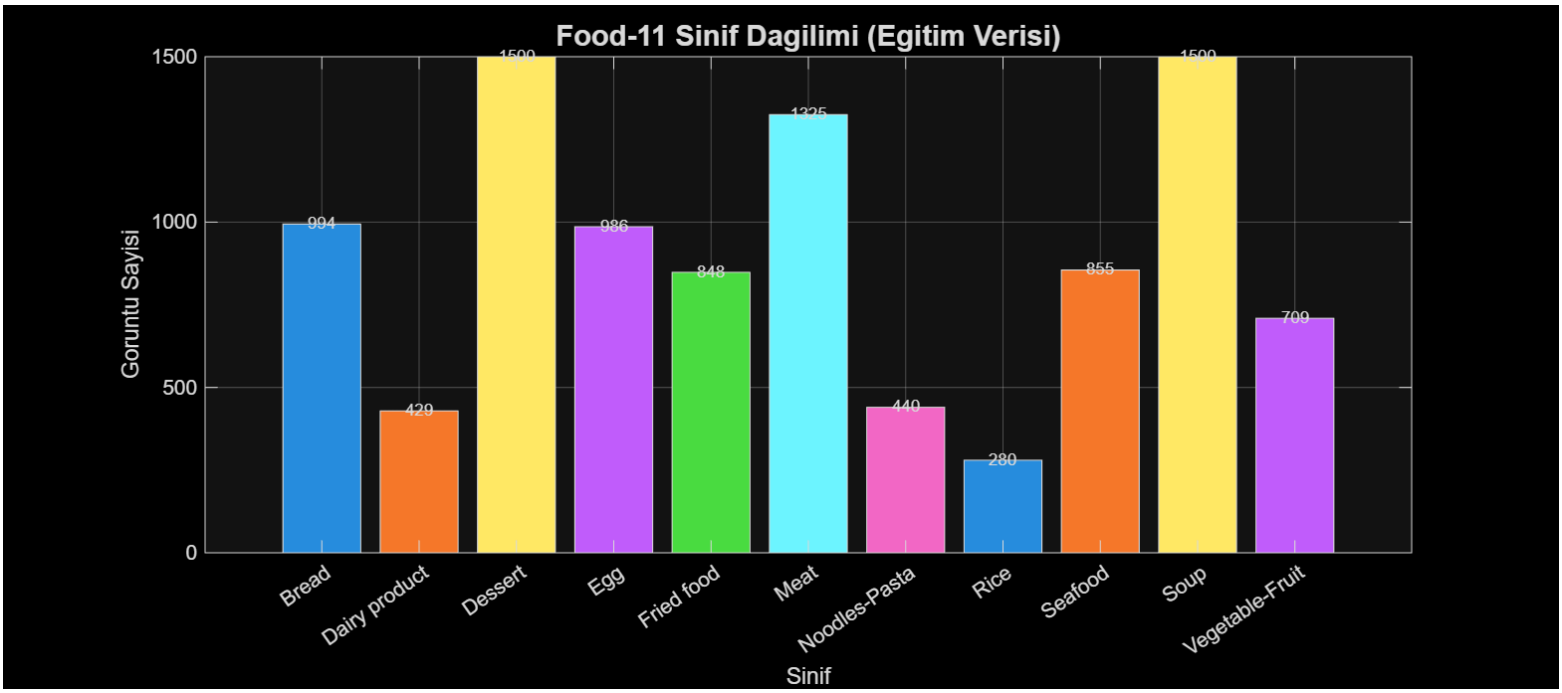
evaluation/ → Eğitim bittikten sonra nihai testin yapıldığı görüntüler

Bu çalışmada Kaggle Food-11 Image Dataset kullanılmıştır. Veri seti, 11 ana yemek kategorisini kapsayan ve üç bölüme ayrılmış görüntülerden oluşmaktadır.

Bölüm	Görüntü Sayısı	
Eğitim (Training)	~9.866	**%80**
Doğrulama (Validation)	~3.430	
Test	~3.347	**%20**

Veri seti şu 11 kategoriden oluşmaktadır: **Bread, Dairy Product, Dessert, Egg, Fried Food, Meat, Noodles-Pasta, Rice, Seafood, Soup, Vegetable-Fruit.**

Veri setinde sınıflar arası örnek sayısı dengesizliği gözlemlenmiştir. Örneğin Soup sınıfı (test setinde 500 örnek) ile Rice sınıfı (test setinde 96 örnek) arasında belirgin bir dengesizlik mevcuttur.



3.2 Sınıf Dengesizliği ve Ağırlıklandırma

Eğitim setindeki dengesizlik, modelin çoğunluk sınıflarına yönelmesine ve azınlık sınıflarında düşük recall değerlerine yol açabilmektedir. Bu sorunu gidermek amacıyla **ağırlıklı çapraz entropi kaybı (weighted cross-entropy loss)** kullanılmıştır. Her sınıfa ait ağırlık aşağıdaki formülle hesaplanmıştır:

$$w_i = k \cdot \frac{N_{toplam}}{N_i}$$

Burada *Ntoplam* toplam örnek sayısını, *k* sınıf sayısını ve *Ni* ilgili sınıftaki örnek sayısını ifade etmektedir.

4. Model Mimarisi

4.1 Transfer Learning Neden Kullanıldı?

Derin öğrenme modellerini sıfırdan eğitmek, milyonlarca görüntü ve büyük hesaplama gücü gerektirmektedir. Transfer learning yaklaşımında ise milyonlarca görüntüyle (ImageNet) önceden eğitilmiş bir modelin ağırlıkları başlangıç noktası olarak alınır. Bu sayede çok daha az veri ve süreyle yüksek başarım elde etmek mümkündür.

4.2 ResNet-50

ResNet-50 (Residual Network — 50 katman), 2015 yılında He ve ark. tarafından tanıtılan ve ImageNet yarışmasında birinci olan derin bir CNN mimarisidir. Temel katkısı, **artık bağlantılar (residual connections)** aracılığıyla gradyan kaybı (vanishing gradient) problemini çözmesi ve çok derin ağların başarıyla eğitilebilmesine imkân tanımasıdır.

4.3 Özelleştirilmiş Çıktı Katmanları

Orijinal ResNet-50'nin 1000 sınıflı ImageNet çıktı katmanları çıkarılarak yerlerine aşağıdaki yapı eklenmiştir:

ResNet-50 Gövdesi (Backbone)

↓

Ortalama Havuzlama (avg_pool)

↓

Dropout Katmanı (%40)

↓

Tam Bağlantılı Katman (11 çıkış)

↓

Softmax (Olasılık Dağılımı)

↓

Sınıflandırma Katmanı (Sınıf Ağırlıklı)

Dropout katmanı, eğitim sırasında nöronların rastgele devre dışı bırakılmasını sağlayarak overfitting'i azaltır.

---

5. Eğitim Stratejisi

5.1 Phase 1 — Özellik Çıkartıcıyı Dondurarak FC Eğitimi (10 Epoch)

Backbone (ResNet-50 evrişim katmanları) tamamen dondurulmuş; yalnızca eklenen Dropout + FC katmanı eğitilmiştir. Bu aşamanın amacı, FC katmanının ağırlıklarını rastgele başlangıç noktasından anlamlı bir bölgeye taşımaktır.

Hiperparametre	**Değer**
Optimizer	Adam
Learning Rate	1×10^{-3}
Epoch	10
Mini-batch	32
Erken Durdurma	Kapalı

Aşama 1 sonunda validation accuracy yaklaşık **%82–84** seviyesine ulaşmıştır.

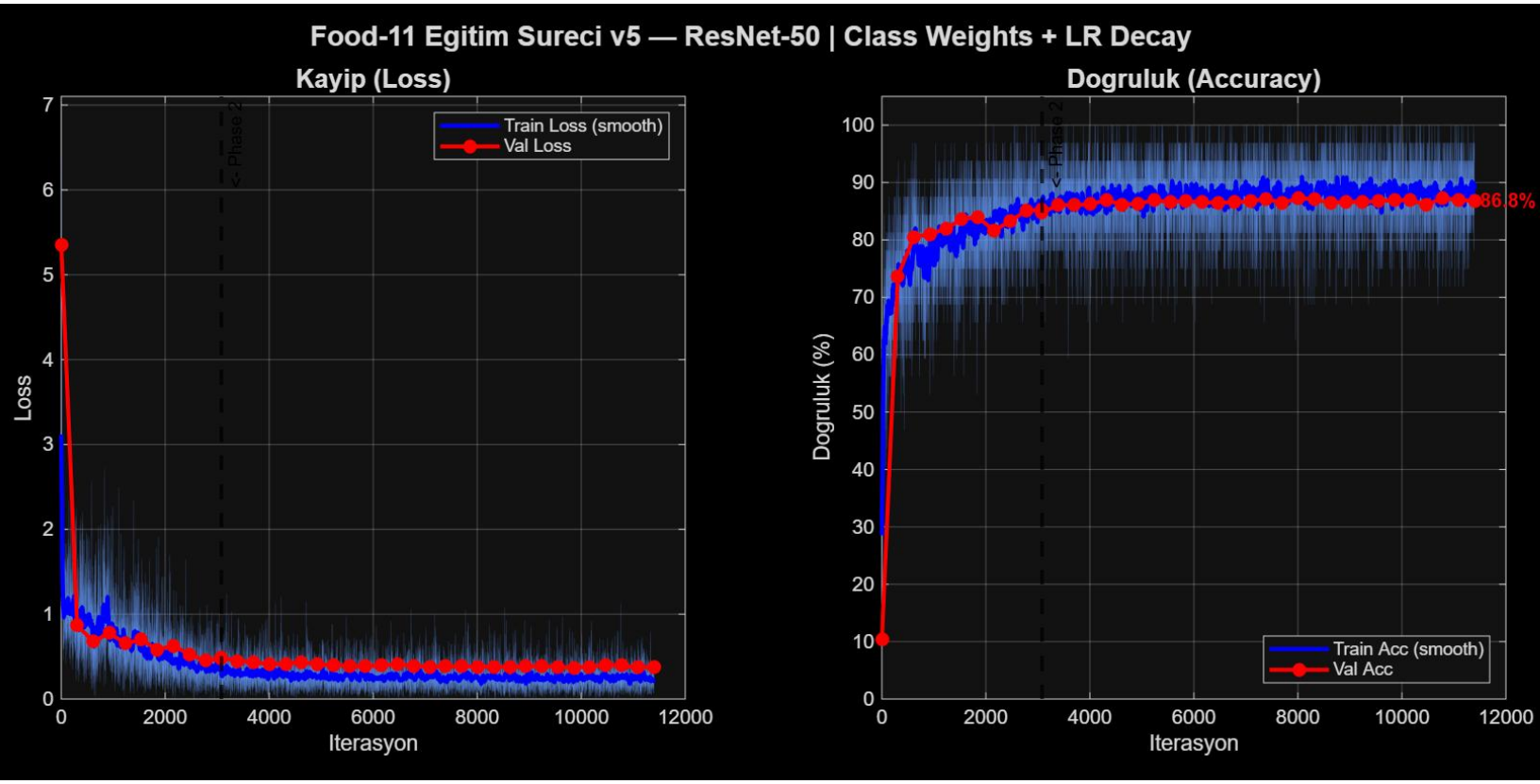
5.2 Phase 2 — Kısmi İnce Ayar (Fine-Tuning) (maks. 40 Epoch)

Aşama 1 tamamlandıktan sonra ResNet-50'nin son iki bloğu (res4 ve res5) farklı öğrenme hızı katsayılarıyla açılmıştır.

Katman Grubu	**LR Faktörü**	**Gerekçe**
res4 + bn4	0.01×	Genel özellikler — küçük güncelleme yeterli
res5 + bn5	0.1×	Sınıfa özgü özellikler — orta güncelleme
fc_food	1×	Yeni katman — tam hızda güncelleme
Diğer katmanlar	0	Donmuş — güncellenmez

Hiperparametre	Değer
Optimizer	SGDM (Momentum = 0.9)
Learning Rate	5×10^{-5}
LR Azaltma	Her 20 epoch'ta 0.3×
Epoch (maks.)	40
Mini-batch	32
Erken Durdurma	ValidationPatience = 5

Validation loss her epoch sonunda ölçülmüş; ardışık 5 epoch boyunca iyileşme gözlemlenmediğinde eğitim otomatik olarak sonlandırılmıştır. Eğitim süreci yaklaşık **~12.000 iterasyon** sonunda tamamlanmış, validation accuracy son noktada **%86,8** olarak gözlemlenmiştir



5.3 Veri Artırma

Eğitim setine yalnızca eğitim verilerine uygulanmak üzere aşağıdaki veri artırma işlemleri uygulanmıştır:

İşlem	Parametre
Yatay çevirme	Rastgele
Döndürme	±30°
Yatay/Dikey ölçekleme	0.85 – 1.15×
Kesme (shear)	±10°
Parlaklık/Kontrast/Doygunluk	±%10

Doğrulama ve test setlerine yalnızca yeniden boyutlandırma (224×224) ve gri→RGB dönüşümü uygulanmıştır.

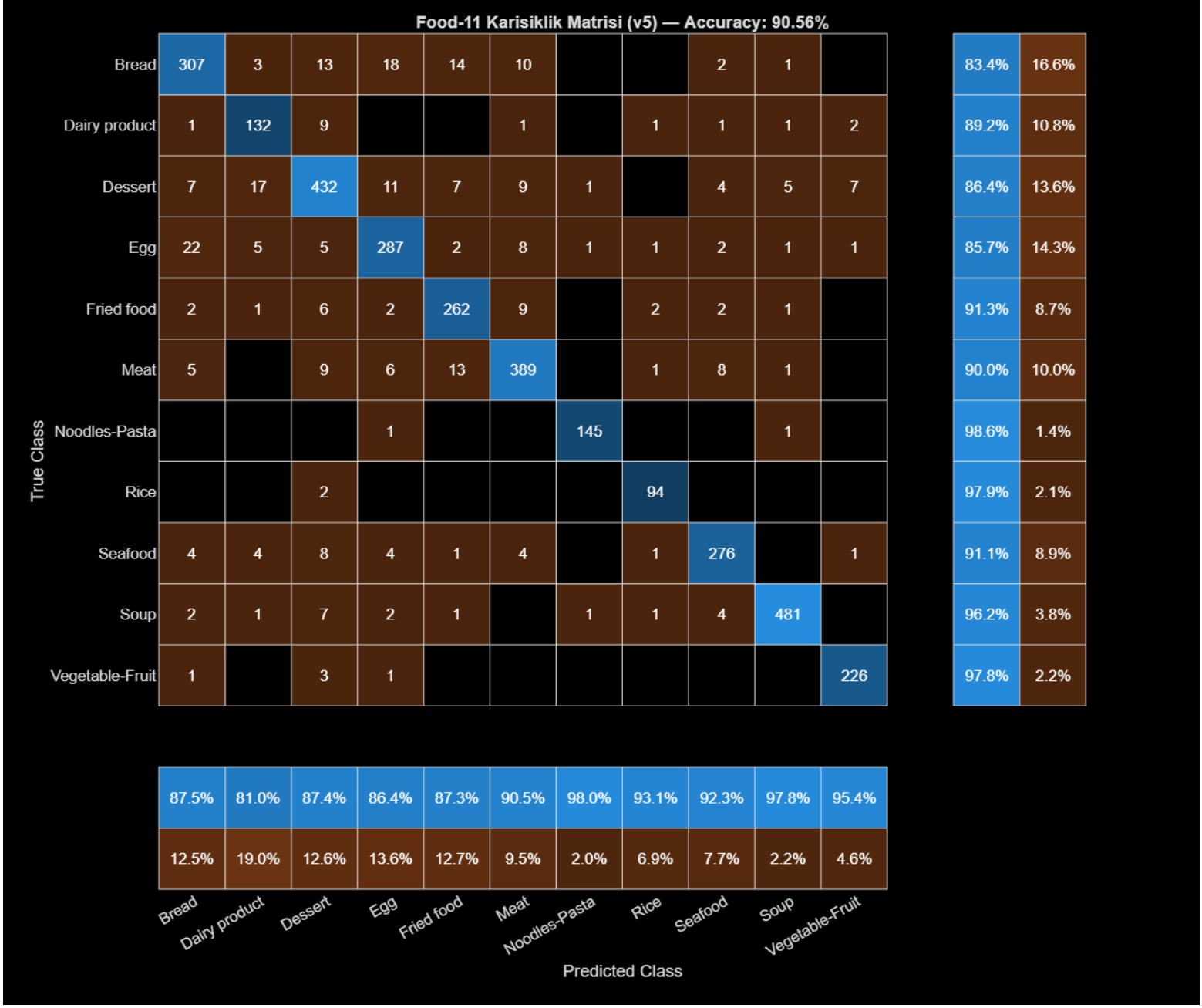
---

6. Deneysel Sonuçlar

6.1 Genel Performans Metrikleri

Metrik	Değer
Test Accuracy	**%90,56**
Makro F1	**%91,1**
Ağırlıklı F1	**%90,6**
Validation Accuracy (son epoch)	**%86,8**

Not: Test accuracy ile validation accuracy arasındaki fark, eğitim sürecinde erken durdurmayı tetikleyen checkpoint modelinin test setindeki gerçek performansını yansıtmaktadır. Test seti hiçbir eğitim kararında kullanılmamıştır.

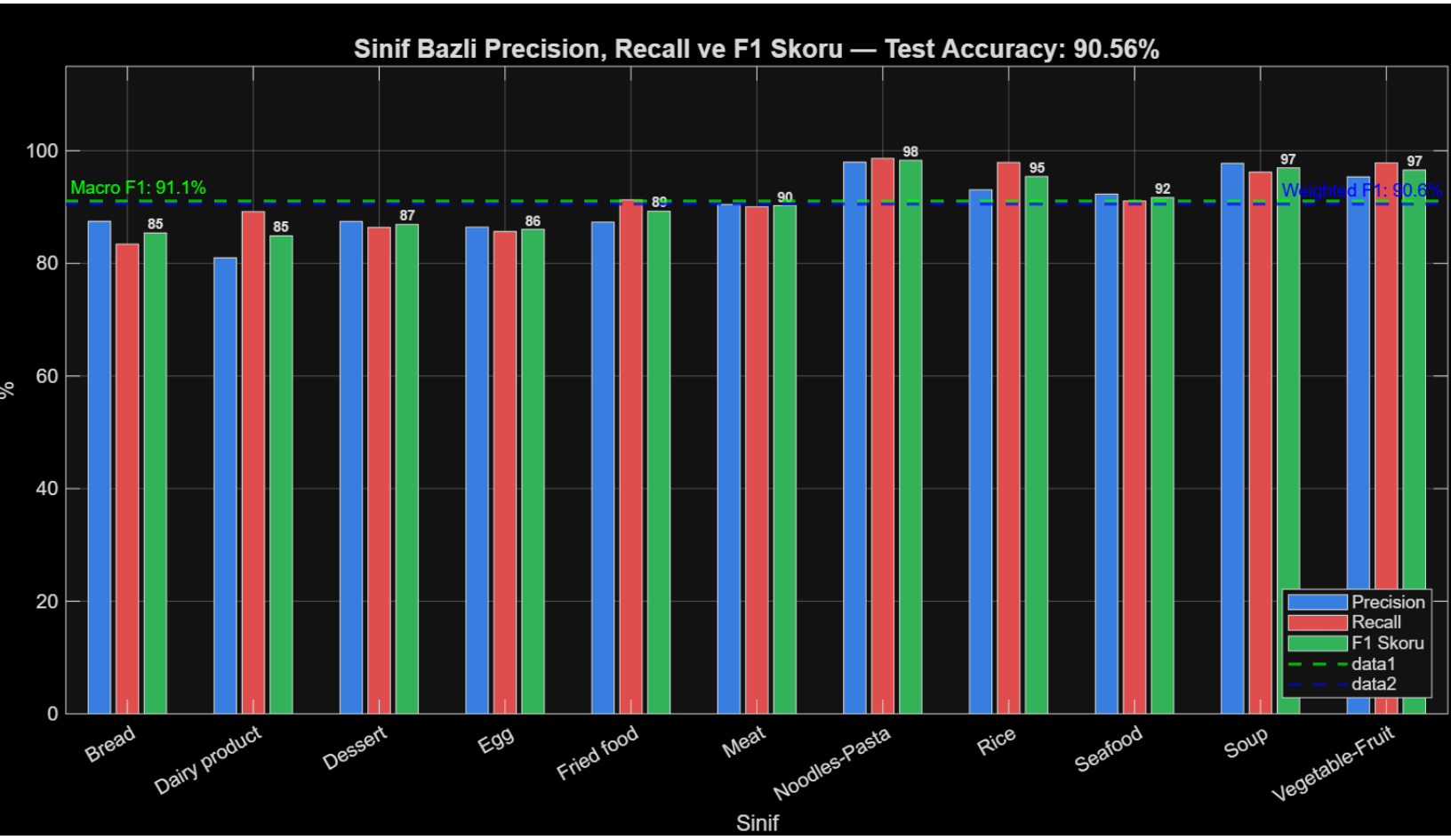


6.2 Sınıf Bazlı Performans

Karmaşıklık matrisi ve F1 skoru grafikleri incelendiğinde:

Sınıf	Precision	Recall	F1	Doğru / Toplam
Bread	%87,5	%83,4	%85	307 / 368
Dairy Product	%81,0	%89,2	%85	132 / 148
Dessert	%87,4	%86,4	%87	432 / 500
Egg	%86,4	%85,7	%86	287 / 335

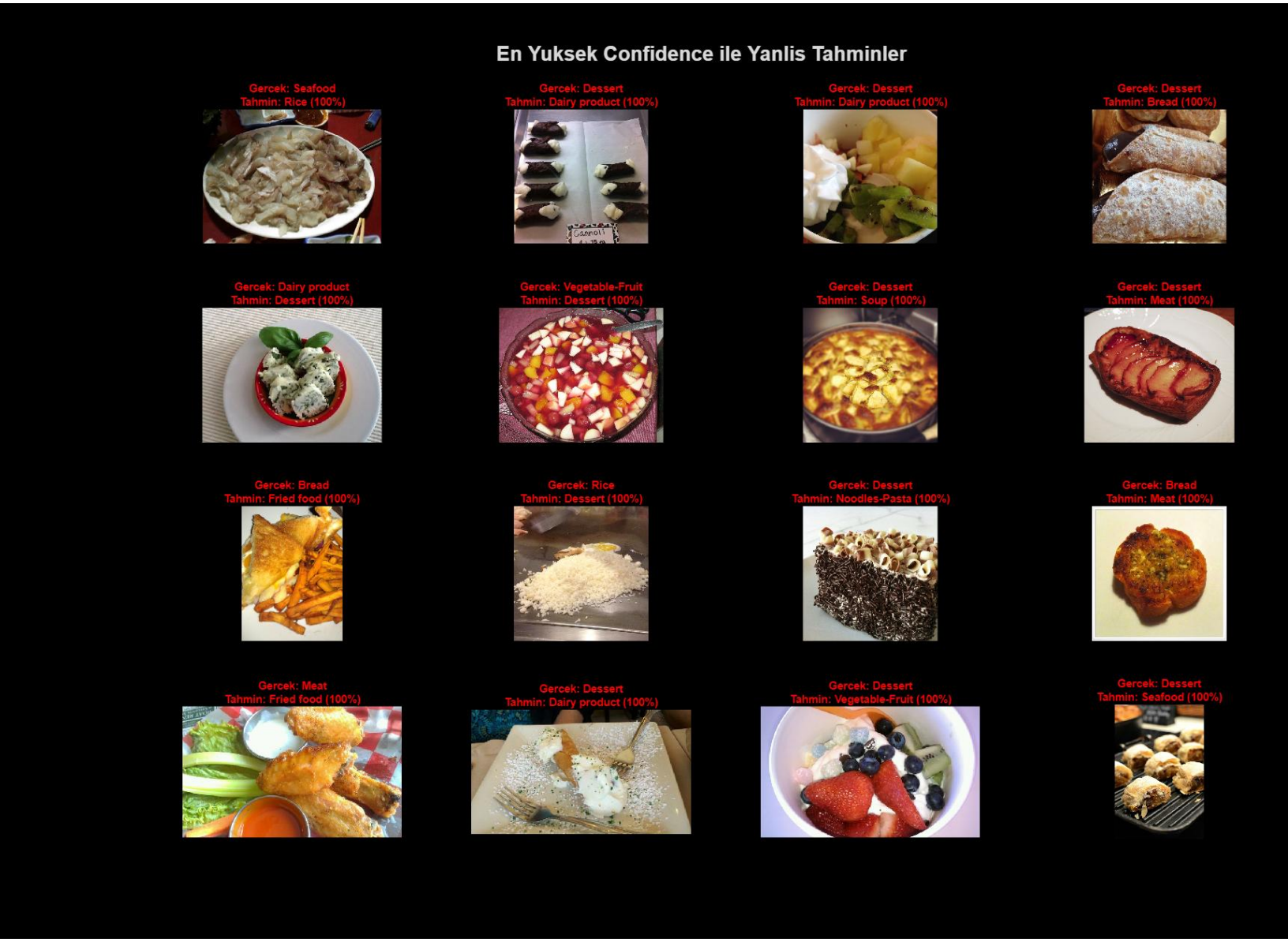
Sınıf	Precision	Recall	F1	Doğru / Toplam
Fried Food	%87,3	%91,3	%89	262 / 287
Meat	%90,5	%90,0	%90	389 / 432
Noodles-Pasta	%98,0	%98,6	%98	145 / 147
Rice	%93,1	%97,9	%95	94 / 96
Seafood	%92,3	%91,1	%92	276 / 303
Soup	%97,8	%96,2	%97	481 / 500
Vegetable-Fruit	%95,4	%97,8	%97	226 / 231



6.4 Düşük Performanslı Sınıflar ve Karışıklık Analizi

Karmaşıklık matrisine göre en fazla sınıflar arası karışıklık şu çiftler arasında yaşanmıştır:

- **Bread ↔ Dessert / Meat:** Bread sınıfından 13 örnek Dessert'e, 18'i Egg'e yanlış sınıflandırılmıştır. Ekmek bazlı tatlılar (örn. krep, gözleme) görsel olarak Dessert ile örtüşmektedir.
- **Egg ↔ Bread:** 22 Egg örneği Bread olarak tahmin edilmiştir. Yumurtalı börek veya tost gibi yemekler bu karışıklığın kaynağıdır.
- **Dessert → Dairy Product / Bread:** Dessert sınıfı en fazla yanlış tahmin alan sınıftır (68 hata). Cheesecake, dondurma gibi süt ürünü içeren tatlılar bu karışıklığı açıklamaktadır.



Yüksek güvenli yanlış tahminler modelin zor örneklerde de kararlı ancak hatalı tahmin ürettiğini göstermektedir. Bu durum veri setindeki görsel örtüşmeden kaynaklanmaktadır, model belirsizliği değil sınıf tanımı belirsizliğidir.

6.5 Eğitim Süreci

Eğitim eğrilerine göre:

- Train loss yaklaşık 2.000. iterasyona kadar hızla düşmüş, ardından kademeli olarak yakınsamıştır.
- Validation loss, train loss'a paralel seyretmiş; belirgin bir aşırı öğrenme gözlemlenmemiştir.
- Train accuracy ~%93-94 seviyesine ulaşırken validation accuracy %86,8'de sabitlenmiştir.

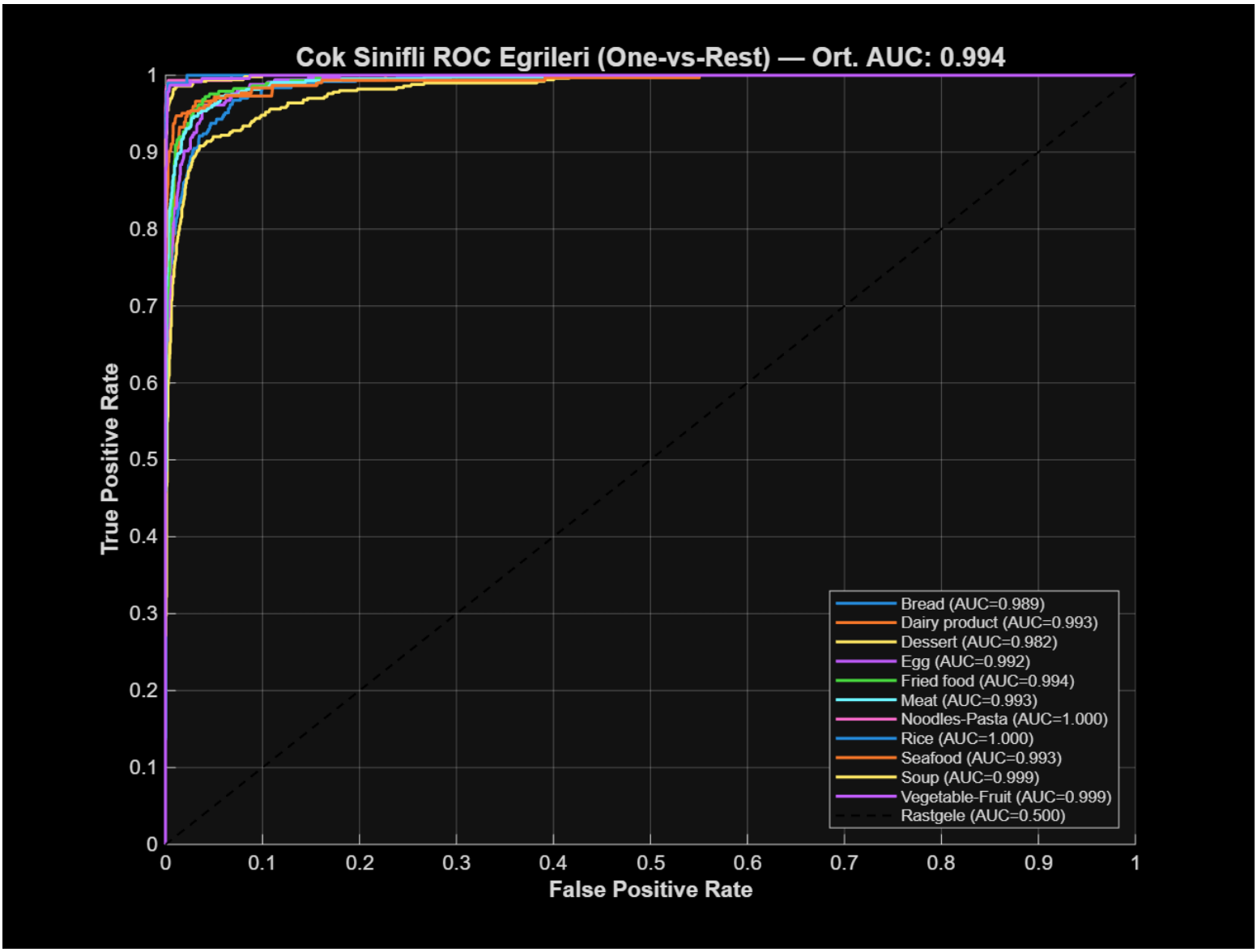
6.6 ROC Eğrileri ve AUC Analizi

Her sınıf için One-vs-Rest yaklaşımıyla hesaplanan ROC eğrileri ve AUC değerleri aşağıdaki tabloda sunulmuştur:

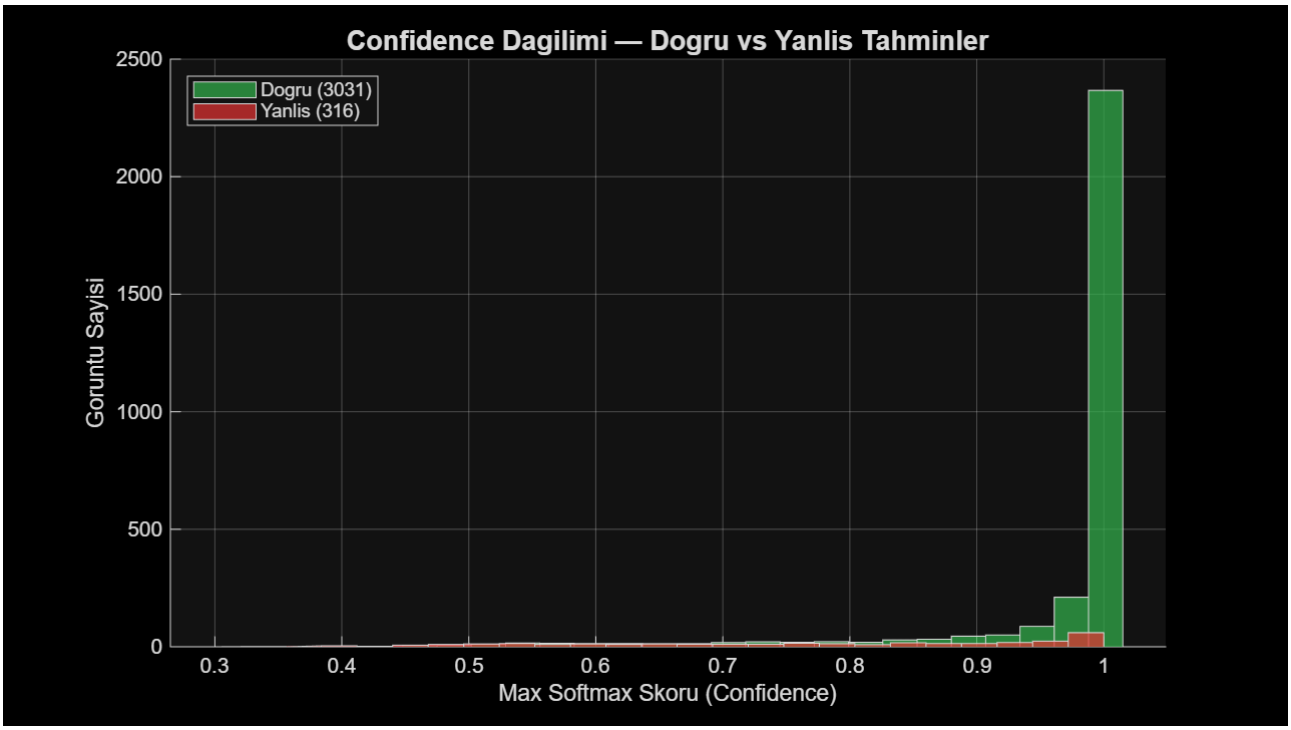
Sınıf	**AUC**
Bread	0.989
Dairy Product	0.993
Dessert	0.982
Egg	0.992
Fried Food	0.994
Meat	0.993
Noodles-Pasta	**1.000**
Rice	**1.000**
Seafood	0.993

Sınıf	AUC
Soup	0.999
Vegetable-Fruit	0.999
Ortalama AUC	**0.994**

Tüm sınıflar 0.98'in üzerinde AUC değeri elde etmiştir. Noodles-Pasta ve Rice sınıfları mükemmel (AUC = 1.000) ayrım kapasitesi sergilemiştir. En düşük AUC değeri Dessert sınıfına (0.982) aittir; bu durum Dessert'in diğer sınıflarla (özellikle Dairy Product ve Bread) görsel örtüşmesinin bir yansımasıdır. Ortalama AUC = 0.994 değeri, modelin tüm eşik değerlerinde son derece güçlü bir ayrım kapasitesine sahip olduğunu ortaya koymaktadır.



6.7 Güven Skoru Dağılımı (Confidence Distribution)



Confidence dağılım grafiği, doğru ve yanlış tahminlerin softmax güven skorlarına göre ayrımını göstermektedir. Test setindeki toplam **3.347 görüntünün 3.031'i (%90,56) doğru, 316'sı (%9,44) yanlış** sınıflandırılmıştır.

Grafikten çıkan başlıca bulgular şunlardır:

- **Doğru tahminlerin büyük çoğunluğu (%0.95–1.00 aralığında)** son derece yüksek güvenle yapılmıştır. Yaklaşık 2.350 doğru tahmin, softmax skoru 0.99'un üzerindedir. Bu, modelin doğru sınıflandırdığı örneklerde kararlı ve güvenli kararlar verdiğini göstermektedir.
- **Yanlış tahminlerin de önemli bir kısmı yüksek güvenle yapılmıştır.** 316 yanlış tahmininin yaklaşık 60–70'i 0.95 üzeri güven skoruna sahiptir. Bu durum, modelin bazı örneklerde hatalı ama emin olduğunu; söz konusu hataların rastgele değil, sistematik sınıf karışıklıklarından (örn. Dessert → Dairy Product) kaynaklandığını işaret etmektedir.
- Düşük güven bölgelerinde (0.3–0.8) her iki grup da oldukça seyrek dağılmıştır. Bu, modelin belirsiz örnekleri nadiren gördüğünü ve güçlü bir karar sınırı öğrendiğini göstermektedir.

6.8 Grad-CAM Açıklanabilirlik Analizi



Grad-CAM (Gradient-weighted Class Activation Mapping), modelin karar verirken görüntünün hangi bölgelerine odaklandığını görselleştiren bir açıklanabilirlik yöntemidir. Isı haritasındaki kırmızı bölgeler yüksek aktivasyonu, mavi bölgeler düşük aktivasyonu temsil etmektedir.

Grad-CAM görselinde, model bir churros görüntüsünü **%99 güvenle Fried Food** olarak doğru sınıflandırmıştır. Isı haritası incelendiğinde:

- **Modelin aktivasyonu churros'un kızarmış, kabarık dokusuna yoğunlaşmıştır.** Bu bölge Fried Food sınıfını tanımlamada en ayırt edici görsel ipucu olan yüzey dokusu ve kahverengi kızarma rengiyle örtüşmektedir.
- Yanında yer alan sos kasesi düşük aktivasyon göstermiştir; model kararı salt bağlam nesnesine değil, ana yiyeceğin tekstürel özelliğine dayandırmıştır.
- Bu örnek, modelin görüntünün tamamına değil, gıdaya özgü anlamlı bölgelere odaklanabildiğini ortaya koymaktadır.

Genel olarak Grad-CAM sonuçları, modelin karar mekanizmasının insan sezgisiyle büyük ölçüde örtüştüğünü ve kara kutu (black-box) davranışının ötesinde yorumlanabilir özellikler öğrendiğini doğrulamaktadır.

---

7. Sonuç

Bu çalışmada Food-11 veri seti üzerinde ResNet-50 tabanlı bir CNN modeli geliştirilmiş ve iki aşamalı transfer learning stratejisiyle eğitilmiştir. Model, test setinde **%90,56 doğruluk**, **Makro F1 %91,1** ve **Ağırlıklı F1 %90,6** değerleri elde etmiştir.

Uygulanan başlıca teknikler ve katkıları:

- **Aşamalı backbone açma:** Önceden öğrenilmiş ağırlıkların korunmasına ve yeni veri setine uyarlanmasına imkân tanımıştır.
- **Sınıf ağırlıklandırması:** Dengesiz veri dağılımının azınlık sınıfları üzerindeki olumsuz etkisi azaltılmıştır.
- **Veri artırma:** Geometrik dönüşümler ve renk pertürbasyonları aracılığıyla modelin genelleme kapasitesi artırılmıştır.
- **Erken durdurma:** Gereksiz hesaplamalar önlenmiş, aşırı öğrenmenin başladığı noktada eğitim sonlandırılmıştır.

8. Eğitim Kodları

```
%% ============================================================
%  Yapay Sinir Ağları Dersi - CNN Yemek Kategori Tahmini
%  MATLAB | ResNet-50 | 2 Aşamalı Transfer Learning | F1, Grad-CAM,
%  Confusion, ROC AUC

clc; clear; close all;
rng(42);

%% ------------------------------------------------------------
%  BÖLÜM 1 – YAPILANDIRMA
% ------------------------------------------------------------
TRAIN_DIR = 'training';
VAL_DIR   = 'validation';
EVAL_DIR  = 'evaluation';

IMG_SIZE  = [224 224 3];

BATCH_P1  = 32;
EPOCHS_P1 = 10;
LR_P1     = 1e-3;

BATCH_P2  = 32;
EPOCHS_P2 = 40;
LR_P2     = 5e-5;

if canUseGPU()
    execution = 'gpu';
    fprintf('[INFO] GPU Aktif: %s\n', gpuDevice().Name);
else
    execution = 'cpu';
    fprintf('[INFO] CPU kullaniliyor.\n');
end

%% ------------------------------------------------------------
%  BÖLÜM 2 – DATASTORE OKUMA
```

```
%  BOLUM 2 – DATASTORE OKUMA
% ─────────────────────────────────────────────────────────

imdsTrain = imageDatastore(TRAIN_DIR, 'IncludeSubfolders', true, 'LabelSource', 'foldernames');
imdsVal   = imageDatastore(VAL_DIR,   'IncludeSubfolders', true, 'LabelSource', 'foldernames');
imdsTest  = imageDatastore(EVAL_DIR,  'IncludeSubfolders', true, 'LabelSource', 'foldernames');

classNames = categories(imdsTrain.Labels);
numClasses = numel(classNames);

labelCounts = countEachLabel(imdsTrain);
N_total     = sum(labelCounts.Count);

fprintf('[INFO] Sinif Sayisi : %d\n', numClasses);
fprintf('[INFO] Train: %d | Val: %d | Test: %d\n', ...
    numel(imdsTrain.Files), numel(imdsVal.Files), numel(imdsTest.Files));
fprintf('\n[INFO] Sinif basi ornek sayilari:\n');
for i = 1:numClasses
    bar_len = round(labelCounts.Count(i) / max(labelCounts.Count) * 40);
    fprintf('  %-20s %4d  %s\n', char(labelCounts.Label(i)), ...
        labelCounts.Count(i), repmat('*', 1, bar_len));
end
fprintf('  Min: %d | Max: %d | Oran: %.1fx\n', ...
    min(labelCounts.Count), max(labelCounts.Count), ...
    max(labelCounts.Count) / min(labelCounts.Count));
```

```
%% ─────────────────────────────────────────────────────────
%  BÖLÜM 3 – CLASS WEIGHTS + AUGMENTATION
% ─────────────────────────────────────────────────────────

orderedW = N_total ./ (numClasses .* labelCounts.Count);
orderedW = (orderedW / sum(orderedW) * numClasses)';

augmenter = imageDataAugmenter( ...
    'RandXReflection', true, ...
    'RandRotation',    [-30, 30], ...
    'RandXScale',      [0.85, 1.15], ...
    'RandYScale',      [0.85, 1.15], ...
    'RandXShear',      [-10, 10], ...
    'RandYShear',      [-10, 10]);

augTrain = augmentedImageDatastore(IMG_SIZE(1:2), imdsTrain, ...
    'DataAugmentation', augmenter, 'ColorPreprocessing', 'gray2rgb');
augTrain = transform(augTrain, @augmentBrightness);

augVal = augmentedImageDatastore(IMG_SIZE(1:2), imdsVal, ...
    'ColorPreprocessing', 'gray2rgb');
```

```
%% ─────────────────────────────────────────────────────────
%  BÖLÜM 4 – MİMARİ (ResNet-50)
% ─────────────────────────────────────────────────────────

net    = resnet50;
lgraph = layerGraph(net);

newDrop = dropoutLayer(0.4,        'Name', 'dropout_food');
newFc   = fullyConnectedLayer(numClasses, 'Name', 'fc_food', ...
              'WeightLearnRateFactor', 10, 'BiasLearnRateFactor', 10);
newSM   = softmaxLayer(            'Name', 'softmax_food');
newCL   = classificationLayer(     'Name', 'output_food', ...
              'Classes', classNames, 'ClassWeights', orderedW);
```

```
lgraph = removeLayers(lgraph, {'fc1000','fc1000_softmax','ClassificationLayer_fc1000'});
lgraph = addLayers(lgraph, [newDrop; newFc; newSM; newCL]);
lgraph = connectLayers(lgraph, 'avg_pool', 'dropout_food');

% Phase 1: Backbone tamamen dondurma
for i = 1:numel(lgraph.Layers)
    L = lgraph.Layers(i);
    if ismember(L.Name, {'fc_food', 'dropout_food'}), continue; end
    if isprop(L, 'WeightLearnRateFactor')
        L.WeightLearnRateFactor = 0;
        L.BiasLearnRateFactor   = 0;
        lgraph = replaceLayer(lgraph, L.Name, L);
    end
end
fprintf('[INFO] ResNet-50 hazir, Phase 1 icin donduruldu.\n');
```

```
%% ─────────────────────────────────────────────────────────
%  BÖLÜM 5 – PHASE 1 EĞİTİMİ
% ─────────────────────────────────────────────────────────

opts_p1 = trainingOptions('adam', ...
    'MaxEpochs',            EPOCHS_P1, ...
    'MiniBatchSize',        BATCH_P1, ...
    'InitialLearnRate',     LR_P1, ...
    'ValidationData',       augVal, ...
    'ValidationFrequency',  floor(numel(imdsTrain.Files) / BATCH_P1), ...
    'ValidationPatience',   Inf, ...
    'Shuffle',              'every-epoch', ...
    'ExecutionEnvironment', execution, ...
    'Plots',                'training-progress', ...
    'Verbose',              true);

[netP1, infoP1] = trainNetwork(augTrain, lgraph, opts_p1);
fprintf('\n[PHASE 1 BITTI]\n');
```

```
%  BÖLÜM 6 – PHASE 2 EĞİTİMİ (res4 + res5)
% ─────────────────────────────────────────────────────────

lgraphP2 = layerGraph(netP1);

for i = 1:numel(lgraphP2.Layers)
    L = lgraphP2.Layers(i);
    if ~isprop(L, 'WeightLearnRateFactor'), continue; end
    if strcmp(L.Name, 'fc_food')
        L.WeightLearnRateFactor = 1;    L.BiasLearnRateFactor = 1;
    elseif contains(L.Name, {'res5', 'bn5'})
        L.WeightLearnRateFactor = 0.1;  L.BiasLearnRateFactor = 0.1;
    elseif contains(L.Name, {'res4', 'bn4'})
        L.WeightLearnRateFactor = 0.01; L.BiasLearnRateFactor = 0.01;
    end
    lgraphP2 = replaceLayer(lgraphP2, L.Name, L);
end

% EARLY STOPPING
% ValidationPatience = 5 → val loss 5 ardışık kontrolde iyileşmezse dur
% ValidationFrequency = iterPerEpoch → her epoch sonunda 1 kez ölçülür
% Her epoch sonunda 1 ölçüm, 5 epoch iyileşme yoksa dur.
iterPerEpoch = floor(numel(imdsTrain.Files) / BATCH_P2);

opts_p2 = trainingOptions('sgdm', ...
    'MaxEpochs',            EPOCHS_P2, ...
    'MiniBatchSize',        BATCH_P2, ...
    'InitialLearnRate',     LR_P2, ...
    'Momentum',             0.9, ...
    'LearnRateSchedule',    'piecewise', ...
```

```
    'LearnRateSchedule',    'piecewise', ...
    'LearnRateDropFactor',  0.3, ...
    'LearnRateDropPeriod',  20, ...
    'ValidationData',       augVal, ...
    'ValidationFrequency',  iterPerEpoch, ...  % Her epoch sonunda 1 kez ölç
    'ValidationPatience',   5, ...             % 5 epoch iyileşme yoksa dur
    'Shuffle',              'every-epoch', ...
    'ExecutionEnvironment', execution, ...
    'Plots',                'training-progress', ...
    'Verbose',              true);

[trainedNet, infoP2] = trainNetwork(augTrain, lgraphP2, opts_p2);

save('food11_resnet50_model_v5.mat', 'trainedNet', 'classNames', ...
     'infoP1', 'infoP2', 'labelCounts');
fprintf('\n[INFO] Model kaydedildi: food11_resnet50_model_v5.mat\n');
```

```
%% ─────────────────────────────────────────────────────
%  BÖLÜM 7 - TAHMİN VE METRİKLER
% ─────────────────────────────────────────────────────

augTest    = augmentedImageDatastore(IMG_SIZE(1:2), imdsTest, 'ColorPreprocessing', 'gray2rgb');
trueLabels = imdsTest.Labels;
scores     = predict(trainedNet, augTest, ...
    'ExecutionEnvironment', execution, 'MiniBatchSize', BATCH_P2);


[~, predIdx]  = max(scores, [], 2);
predLabels    = categorical(classNames(predIdx));

accuracy      = mean(predLabels == trueLabels) * 100;
p1FinalAcc    = max(infoP1.ValidationAccuracy(~isnan(infoP1.ValidationAccuracy)));


[maxConf, ~]  = max(scores, [], 2);
correctMask   = (predLabels == trueLabels);
% Per-class metrikler (Precision, Recall, F1)
precArr = zeros(1, numClasses);
recArr  = zeros(1, numClasses);
f1Arr   = zeros(1, numClasses);
nArr    = zeros(1, numClasses);

fprintf('\n%-20s  %6s  %7s  %7s  %7s\n', 'Sinif', 'N_test', 'Prec%', 'Rec%', 'F1%');
fprintf('%s\n', repmat('-', 1, 57));
for i = 1:numClasses
    cls = classNames{i};
    tp  = sum(predLabels == cls & trueLabels == cls);
    fp  = sum(predLabels == cls & trueLabels ~= cls);
    fn  = sum(predLabels ~= cls & trueLabels == cls);
    n_c = sum(trueLabels == cls);
    p   = 100 * tp / max(tp + fp, 1);
    r   = 100 * tp / max(tp + fn, 1);
    f1  = 2 * p * r / max(p + r, 1e-6);
    precArr(i) = p;  recArr(i) = r;  f1Arr(i) = f1;  nArr(i) = n_c;
    fprintf('%-20s  %6d  %7.1f  %7.1f  %7.1f\n', char(cls), n_c, p, r, f1);
end

macroF1    = mean(f1Arr);
weightedF1 = sum(f1Arr .* nArr) / sum(nArr);
```

```
%% ─────────────────────────────────────────────────────────
%  BÖLÜM 8 – GRAFİKLER
% ─────────────────────────────────────────────────────────

% ── 8a. Sınıf Dağılımı ──────────────────────────────────
figure('Name', 'Sinif Dagilimi', 'Position', [50 50 1000 440]);
b0 = bar(1:numClasses, labelCounts.Count, 'FaceColor', 'flat');
b0.CData = lines(numClasses);
set(gca, 'XTick', 1:numClasses, 'XTickLabel', classNames);
xtickangle(35); grid on;
title('Food-11 Sinif Dagilimi (Egitim Verisi)', 'FontSize', 14, 'FontWeight', 'bold');
xlabel('Sinif'); ylabel('Goruntu Sayisi');
for i = 1:numClasses
    text(i, labelCounts.Count(i) + 4, num2str(labelCounts.Count(i)), ...
         'HorizontalAlignment', 'center', 'FontSize', 8);
end
saveas(gcf, 'food11_class_distribution_v5.png');
fprintf('[KAYIT] food11_class_distribution_v5.png\n');

% ── 8b. Confusion Matrix ────────────────────────────────
figure('Name', 'Confusion Matrix', 'Position', [50 50 820 720]);
cm = confusionchart(trueLabels, predLabels, ...
    'Title', sprintf('Food-11 Karisiklik Matrisi (v5) — Accuracy: %.2f%%', accuracy), ...
    'RowSummary', 'row-normalized', 'ColumnSummary', 'column-normalized');
cm.FontSize = 8;
saveas(gcf, 'food11_confusion_matrix_v5.png');
fprintf('[KAYIT] food11_confusion_matrix_v5.png\n');

% ── 8c. Training Curves ─────────────────────────────────
nP1        = numel(infoP1.TrainingLoss);
allTrLoss  = [infoP1.TrainingLoss,       infoP2.TrainingLoss];
allTrAcc   = [infoP1.TrainingAccuracy,   infoP2.TrainingAccuracy];
allValLoss = [infoP1.ValidationLoss,     infoP2.ValidationLoss];
allValAcc  = [infoP1.ValidationAccuracy, infoP2.ValidationAccuracy];
N          = numel(allTrLoss);
vLossIdx   = find(~isnan(allValLoss));
vAccIdx    = find(~isnan(allValAcc));
smTrLoss   = movmean(allTrLoss, 30);
smTrAcc    = movmean(allTrAcc,  30);

figure('Name', 'Training Curves', 'Position', [100 100 1250 520]);

subplot(1, 2, 1); hold on; box on;
plot(1:N, allTrLoss, 'Color', [0.4 0.6 1 0.15], 'LineWidth', 0.5);
h1 = plot(1:N, smTrLoss, 'b-', 'LineWidth', 2.2, 'DisplayName', 'Train Loss (smooth)');
h2 = plot(vLossIdx, allValLoss(vLossIdx), 'r-o', 'LineWidth', 2, 'MarkerSize', 5, ...
          'MarkerFaceColor', 'r', 'DisplayName', 'Val Loss');
xline(nP1, 'k--', 'LineWidth', 1.8, 'Label', '<- Phase 2', 'FontSize', 9, ...
      'LabelHorizontalAlignment', 'right');
xlabel('Iterasyon'); ylabel('Loss');
title('Kayip (Loss)', 'FontSize', 13, 'FontWeight', 'bold');
legend([h1 h2], 'Location', 'northeast'); grid on;
ylim([0 max(allTrLoss) * 1.1]); hold off;
```

```
subplot(1, 2, 2); hold on; box on;
plot(1:N, allTrAcc, 'Color', [0.4 0.6 1 0.15], 'LineWidth', 0.5);
h3 = plot(1:N, smTrAcc, 'b-', 'LineWidth', 2.2, 'DisplayName', 'Train Acc (smooth)');
h4 = plot(vAccIdx, allValAcc(vAccIdx), 'r-o', 'LineWidth', 2, 'MarkerSize', 5, ...
          'MarkerFaceColor', 'r', 'DisplayName', 'Val Acc');
xline(nP1, 'k--', 'LineWidth', 1.8, 'Label', '<- Phase 2', 'FontSize', 9, ...
      'LabelHorizontalAlignment', 'right');
finalValAcc = allValAcc(vAccIdx(end));
text(N, finalValAcc, sprintf(' %.1f%%', finalValAcc), ...
     'FontSize', 10, 'FontWeight', 'bold', 'Color', 'r');
xlabel('Iterasyon'); ylabel('Dogruluk (%)');
title('Dogruluk (Accuracy)', 'FontSize', 13, 'FontWeight', 'bold');
legend([h3 h4], 'Location', 'southeast'); grid on; ylim([0 105]); hold off;

sgtitle('Food-11 Egitim Sureci v5 — ResNet-50 | Class Weights + LR Decay', ...
        'FontSize', 14, 'FontWeight', 'bold');
saveas(gcf, 'food11_training_curves_v5.png');
fprintf('[KAYIT] food11_training_curves_v5.png\n');

% ── 8d. F1 Skoru Grafiği ──────────────────────────
% Makro F1 ve Weighted F1 yatay referans çizgisi olarak ekleme
figure('Name', 'F1 Skoru', 'Position', [100 100 1200 560]);

barData = [precArr(:), recArr(:), f1Arr(:)];
bh = bar(barData, 'grouped');
bh(1).FaceColor = [0.22 0.49 0.88];   % Mavi  — Precision
bh(2).FaceColor = [0.88 0.30 0.30];   % Kırmızı — Recall
bh(3).FaceColor = [0.20 0.70 0.35];   % Yeşil — F1

set(gca, 'XTick', 1:numClasses, 'XTickLabel', classNames);
xtickangle(30);
legend({'Precision', 'Recall', 'F1 Skoru'}, 'Location', 'southeast', 'FontSize', 9);
title(sprintf('Sinif Bazli Precision, Recall ve F1 Skoru — Test Accuracy: %.2f%%', accuracy), ...
      'FontSize', 13, 'FontWeight', 'bold');
xlabel('Sinif'); ylabel('%'); grid on; ylim([0 115]);
```

```
% Makro F1 ve Weighted F1 referans çizgileri
yline(macroF1,    'g--', sprintf('Macro F1: %.1f%%',    macroF1),    ...
      'LineWidth', 1.5, 'FontSize', 9, 'LabelHorizontalAlignment', 'left');
yline(weightedF1, 'b--', sprintf('Weighted F1: %.1f%%', weightedF1), ...
      'LineWidth', 1.5, 'FontSize', 9, 'LabelHorizontalAlignment', 'right');

% Her bar grubunun F1 değerini üstüne yazdırma
for i = 1:numClasses
    text(i + bh(3).XOffset, f1Arr(i) + 1.5, sprintf('%.0f', f1Arr(i)), ...
         'HorizontalAlignment', 'center', 'FontSize', 7, 'FontWeight', 'bold');
end

saveas(gcf, 'food11_f1_scores_v5.png');
fprintf('[KAYIT] food11_f1_scores_v5.png\n');
```

```
% ── 8e. Per-Class Precision & Recall ──────────────────────
figure('Name', 'Per-Class Performance', 'Position', [100 100 1200 520]);
barData2 = [precArr(:), recArr(:)];
bh2 = bar(barData2, 'grouped');
bh2(1).FaceColor = [0.2 0.5 0.9];
bh2(2).FaceColor = [0.9 0.3 0.3];
set(gca, 'XTick', 1:numClasses, 'XTickLabel', classNames);
xtickangle(30);
legend({'Precision', 'Recall'}, 'Location', 'southeast');
title(sprintf('Sinif Bazli Precision & Recall — Accuracy: %.2f%%', accuracy), ...
      'FontSize', 13, 'FontWeight', 'bold');
xlabel('Sinif'); ylabel('%'); grid on; ylim([0 112]);
yline(accuracy, 'k--', sprintf('Avg %.1f%%', accuracy), 'LineWidth', 1.5, 'FontSize', 9);
saveas(gcf, 'food11_perclass_v5.png');
fprintf('[KAYIT] food11_perclass_v5.png\n');

% ── 8f. Confidence Dağılımı ──────────────────────────────
figure('Name', 'Confidence Distribution', 'Position', [100 100 820 460]);
hold on;
histogram(maxConf(correctMask),  25, 'FaceColor', [0.2 0.7 0.3], 'FaceAlpha', 0.7, ...
          'DisplayName', sprintf('Dogru (%d)',  sum(correctMask)));
histogram(maxConf(~correctMask), 25, 'FaceColor', [0.9 0.2 0.2], 'FaceAlpha', 0.7, ...
          'DisplayName', sprintf('Yanlis (%d)', sum(~correctMask)));
xlabel('Max Softmax Skoru (Confidence)'); ylabel('Goruntu Sayisi');
title('Confidence Dagilimi — Dogru vs Yanlis Tahminler', ...
      'FontSize', 13, 'FontWeight', 'bold');
legend('Location', 'northwest'); grid on; hold off;
saveas(gcf, 'food11_confidence_v5.png');
fprintf('[KAYIT] food11_confidence_v5.png\n');

% ── 8g. En Çok Yanılan Örnekler ──────────────────────────
imdsTest2  = imageDatastore(EVAL_DIR, 'IncludeSubfolders', true, 'LabelSource', 'foldernames');
allFiles   = imdsTest2.Files;
wrongMask  = ~correctMask;
wrongFiles = allFiles(wrongMask);
wrongTrue  = trueLabels(wrongMask);
wrongPred  = predLabels(wrongMask);
wrongConf  = maxConf(wrongMask);
[~, sortW] = sort(wrongConf, 'descend');
showN      = min(16, numel(sortW));

figure('Name', 'En Cok Yanilan Ornekler', 'Position', [50 50 1200 800]);
for k = 1:showN
    idx = sortW(k);
    img_w = imread(wrongFiles{idx});
    subplot(4, 4, k);
    imshow(img_w);
    title(sprintf('Gercek: %s\nTahmin: %s (%.0f%%)', ...
        char(wrongTrue(idx)), char(wrongPred(idx)), wrongConf(idx) * 100), ...
        'FontSize', 7, 'Color', 'r');
end
sgtitle('En Yuksek Confidence ile Yanlis Tahminler', 'FontSize', 13, 'FontWeight', 'bold');
saveas(gcf, 'food11_worst_predictions_v5.png');
fprintf('[KAYIT] food11_worst_predictions_v5.png\n');
```

```
%  BÖLÜM 9 – GRAD-CAM
% ─────────────────────────────────────────────────────────────

correctFiles  = imdsTest.Files(correctMask);
sampleImgPath = correctFiles{randi(length(correctFiles))};

img_gc = imread(sampleImgPath);
img_gc = imresize(img_gc, [224 224]);
if size(img_gc, 3) == 1, img_gc = repmat(img_gc, [1 1 3]); end

[pred_class, score_single] = classify(trainedNet, img_gc);

scoreMap = gradCAM(trainedNet, img_gc, pred_class);

figure('Name', 'Grad-CAM Analizi', 'Position', [100 100 800 400]);

subplot(1, 2, 1);
imshow(img_gc);
title('Orijinal Goruntu', 'FontSize', 12);

subplot(1, 2, 2);
imshow(img_gc); hold on;
scoreMapResized = imresize(scoreMap, [size(img_gc,1) size(img_gc,2)]);
h_cam = imagesc([1 size(img_gc,2)], [1 size(img_gc,1)], scoreMapResized);
h_cam.AlphaData = 0.5;
colormap(gca, jet);
colorbar;
title(sprintf('Agin Odaklandigi Bolge\nTahmin: %s (%.0f%%)', ...
    char(pred_class), max(score_single)*100), 'FontSize', 11);
hold off;
saveas(gcf, 'food11_gradcam_analysis_v5.png');
fprintf('[KAYIT] food11_gradcam_analysis_v5.png\n');
%% ─────────────────────────────────────────────────────────────
%  BÖLÜM 10 – ROC EĞRİSİ ve AUC
% ─────────────────────────────────────────────────────────────

scoreMat   = scores;
aucValues  = zeros(numClasses, 1);
colors_roc = lines(numClasses);

figure('Name', 'ROC Curves', 'Position', [100 100 820 620]);
hold on; grid on;

for i = 1:numClasses
    binaryTrue = (trueLabels == classNames{i});
    [X, Y, ~, AUC] = perfcurve(binaryTrue, scoreMat(:, i), true);
    aucValues(i)   = AUC;
    plot(X, Y, 'Color', colors_roc(i,:), 'LineWidth', 1.5, ...
        'DisplayName', sprintf('%s (AUC=%.3f)', classNames{i}, AUC));
end

plot([0 1], [0 1], 'k--', 'LineWidth', 1, 'DisplayName', 'Rastgele (AUC=0.500)');
meanAUC = mean(aucValues);
title(sprintf('Cok Sinifli ROC Egrileri (One-vs-Rest) – Ort. AUC: %.3f', meanAUC),
      'FontSize', 13, 'FontWeight', 'bold');
xlabel('False Positive Rate', 'FontWeight', 'bold');
ylabel('True Positive Rate', 'FontWeight', 'bold');
legend('Location', 'southeast', 'FontSize', 8);
hold off;
saveas(gcf, 'food11_roc_curves_v5.png');
```

```
% AUC tablosu
fprintf('\nSinif bazli AUC degerleri:\n');
fprintf('%-20s  AUC\n', 'Sinif');
fprintf('%s\n', repmat('-', 1, 28));
for i = 1:numClasses
    fprintf('%-20s  %.4f\n', char(classNames{i}), aucValues(i));
end
fprintf('Ortalama AUC: %.4f\n', meanAUC);

fprintf('\n[TUM ISLEMLER BASARIYLA TAMAMLANDI]\n');
fprintf('Kaydedilen dosyalar:\n');
fprintf('  food11_resnet50_model_v5.mat\n');
fprintf('  food11_class_distribution_v5.png\n');
fprintf('  food11_confusion_matrix_v5.png\n');
fprintf('  food11_training_curves_v5.png\n');
fprintf('  food11_f1_scores_v5.png          (YENİ)\n');
fprintf('  food11_perclass_v5.png\n');
fprintf('  food11_confidence_v5.png\n');
fprintf('  food11_worst_predictions_v5.png\n');
fprintf('  food11_gradcam_analysis_v5.png\n');
fprintf('  food11_roc_curves_v5.png\n');
```

```
%% ─────────────────────────────────────────────────────────
%  YARDIMCI FONKSİYON
% ─────────────────────────────────────────────────────────
function data = augmentBrightness(data)
% Renk/parlaklık varyasyonu (jitterColorHSV ile)
% transform() ile augmentedImageDatastore'a eklenir.
% data.input{1}: [H W C] veya [H W C N] formatında görüntü
    img = data.input{1};
    img = jitterColorHSV(img, ...
        'Hue',        0.05, ...
        'Saturation', 0.10, ...
        'Brightness', 0.10, ...
        'Contrast',   0.10);
    data.input{1} = img;
end
```

Kaynaklar

1. Kaggle Food-11 Image Dataset — https://www.kaggle.com/datasets/trolukovich/food11-image-dataset?select=validation
2. He, K., Zhang, X., Ren, S., & Sun, J. (2016). Deep Residual Learning for Image Recognition. *CVPR 2016*.
3. Selvaraju, R. R., et al. (2017). Grad-CAM: Visual Explanations from Deep Networks via Gradient-based Localization. *ICCV 2017*.
4. Pan, S. J., & Yang, Q. (2010). A Survey on Transfer Learning. *IEEE Transactions on Knowledge and Data Engineering*.
5. Krizhevsky, A., Sutskever, I., & Hinton, G. (2012). ImageNet Classification with Deep Convolutional Neural Networks. *NeurIPS 2012*.
6. MathWorks. (2024). *Deep Learning Toolbox User's Guide*. The MathWorks, Inc.